

Guía de instalación: Visual Studio Code para Python

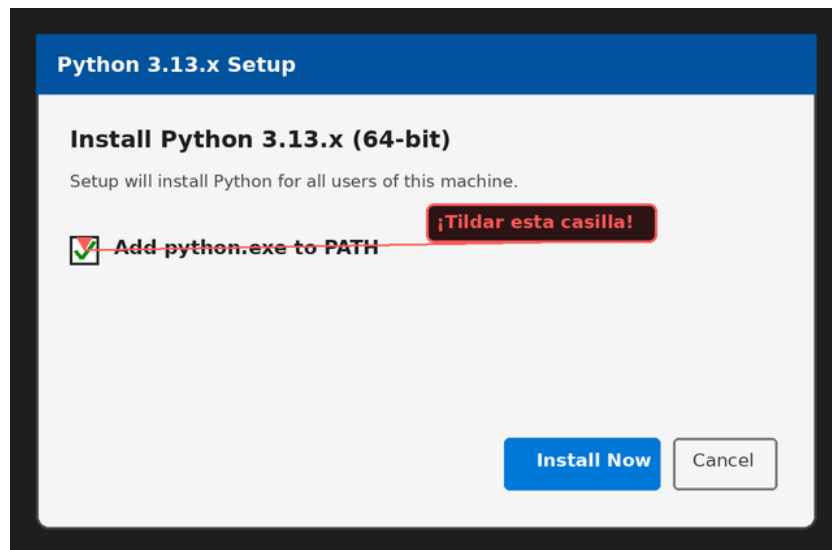
Entorno mínimo, rápido y portable — funciona en cualquier PC (Windows, Linux o macOS)

Esta guía instala solamente lo necesario para programar en Python con autocompletado, linter y formateo automático de código. No se instalan extensiones de más: cuanto más liviano el entorno, más rápido arranca VS Code en cualquier máquina, incluso en notebooks modestas.

Nota: las imágenes de esta guía son diagramas ilustrativos del entorno de VS Code, no capturas de pantalla reales. Los nombres de botones, menús y campos son exactamente los que van a ver en la aplicación real.

Paso 1 — Instalar Python

Descargar el instalador desde python.org/downloads y ejecutarlo. En la primera pantalla del instalador, **es fundamental** tildar la casilla “Add python.exe to PATH” antes de darle a “Install Now”. Si no se tilda, VS Code después no va a poder encontrar el intérprete de Python automáticamente.



Instalador de Python — tildar “Add python.exe to PATH”.

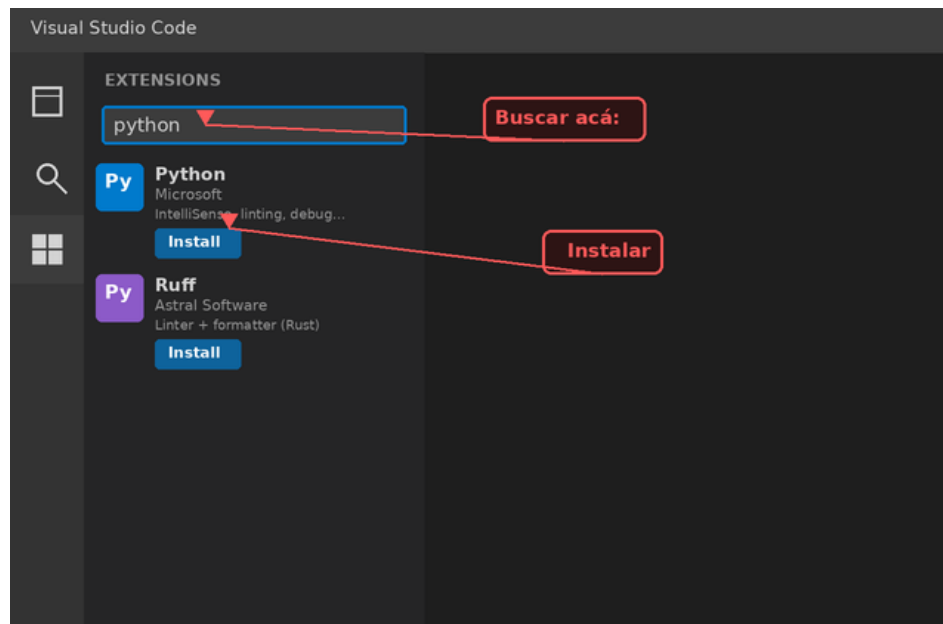
Paso 2 — Instalar Visual Studio Code

Descargar el instalador desde code.visualstudio.com y completar la instalación con las opciones por defecto. No hace falta tildar ninguna opción especial.

Paso 3 — Instalar las extensiones necesarias

Abrir VS Code y hacer clic en el ícono de **Extensions** en la barra lateral izquierda (o presionar Ctrl+Shift+X). Buscar e instalar estas dos extensiones:

Python (Microsoft)	Da soporte de Python: ejecución, debugging y autocompletado inteligente (instala Pylance automáticamente).
Ruff (Astral Software)	Linter + formateador todo en uno, escrito en Rust: extremadamente rápido y liviano, incluso en PCs viejas. Reemplaza Pylint, Flake8, Black y autopep8.



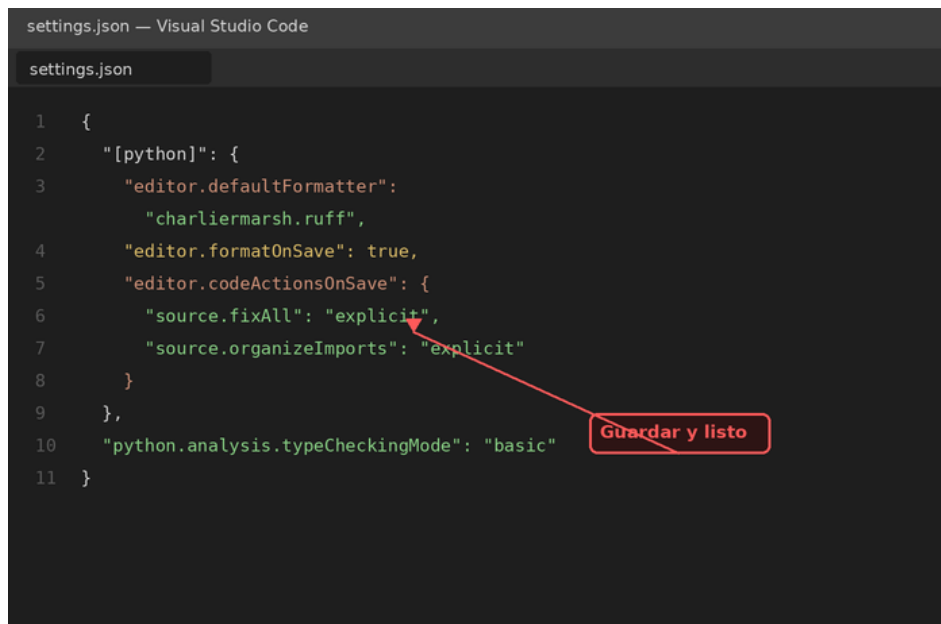
Panel de Extensions: buscar "python" y "ruff", e instalar cada una.

No es necesario instalar nada más (ni Pylint, ni Black, ni Jupyter, ni temas). Cada extensión de más suma tiempo de arranque.

Paso 4 — Configurar el formateo y linting automático

Abrir la paleta de comandos con Ctrl+Shift+P, escribir "Preferences: Open User Settings (JSON)" y pegar esta configuración:

```
{
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
    "editor.formatOnSave": true,
    "editor.codeActionsOnSave": {
      "source.fixAll": "explicit",
      "source.organizeImports": "explicit"
    }
  },
  "python.analysis.typeCheckingMode": "basic"
}
```



```
1 {
2   "[python]": {
3     "editor.defaultFormatter":
4       "charliermarsh.ruff",
5     "editor.formatOnSave": true,
6     "editor.codeActionsOnSave": {
7       "source.fixAll": "explicit",
8       "source.organizeImports": "explicit"
9     },
10    "python.analysis.typeCheckingMode": "basic"
11  }
```

settings.json con formateo automático de Ruff al guardar.

Esto hace que cada vez que se guarde un archivo .py, Ruff lo formatee automáticamente y corrija errores simples (como imports sin usar), y Pylance revise tipos en modo básico sin ser demasiado estricto para un curso introductorio.

Paso 5 — Verificar que todo funciona

Crear un archivo nuevo llamado **hola.py**, escribir algo de código de prueba (por ejemplo, con un import que no se use) y guardar con Ctrl+S. Ruff debería marcar el warning en el panel “Problems” de abajo.



```
1 import os
2
3 def saludar(nombre):
4     print(f"Hola, {nombre}!")
5
6 saludar("mundo")
```

PROBLEMS

⚠ 'os' importado pero no utilizado
Ruff (F401) hola.py [Ln 1, Col 8]

Ruff detecta automáticamente el import sin usar en el panel Problems.

Con estos 5 pasos el entorno queda listo: autocompletado (Pylance), linter y formateo automático (Ruff), sin instalar nada de más. Funciona igual de bien en una PC nueva que en una notebook vieja.